Distilling Magic States in the Bicycle Architecture

Shifan Xu*, Kun Liu†, Patrick Rall‡, Zhiyang He§, Yongshan Ding*†
*Department of Applied Physics, Yale University, New Haven, CT 06511, USA
†Department of Computer Science, Yale University, New Haven, CT 06511, USA
‡IBM Quantum, IBM Research, Cambridge, MA 02142, USA
§Department of Mathematics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

***Abstract*—Magic State Distillation is considered to be one of the promising methods for supplying the non-Clifford resources required to achieve universal fault tolerance. Conventional MSD protocols implemented in surface codes often require multiple code blocks and lattice surgery rounds, resulting in substantial qubit overhead, especially at low target error rates.**

In this work, we present practical magic state distillation factories on Bivariate Bicycle (BB) codes that execute Pauli-measurement-based Clifford circuits inside a single BB code block. We formulate distillation circuit design as a joint optimization of logical qubit mapping, gate scheduling, measurement nativization, and protocol compression via qubit recycling. Based on detailed resource analysis and simulations, our BB factories have space-time volume comparable to that of leading distillation factories while delivering lower target error at a smaller qubit footprint, and are particularly compelling as second-round distillers following magic state cultivations.

***Index Terms*—Magic state distillation, quantum LDPC codes, fault-tolerant quantum computing**

I. Introduction

Building large-scale quantum computers, which can realize transformative applications such as factoring [1], requires the use of quantum error correction to suppress physical noise and enable universal, fault-tolerant quantum computation (FTQC) [2], [3]. To protect quantum information from decoherence, foundational works [4]–[7] showed that we can encode information in *stabilizer codes*, and perform repeated quantum measurements and classical decoding to correct from arbitrary physical errors. These schemes enabled the development of threshold theorems for FTQC [8], which states that arbitrarily large quantum computation can be realized through QEC, assuming that physical error rates are below a constant threshold. Among the many codes developed, the surface code [9]–[11] has been at the center of QEC research for the past two decades due to its promising practical performance, including low connectivity requirement, high threshold and fast decoding algorithms. Despite these advantages, surface code incurs a significant space overhead in realizing FTQC: factoring 2048-bit integers in surface code architectures uses physical qubits on the scale of millions [12], [13]. More recently, significant research has studied quantum low-density parity-check (LDPC) codes, which can realize FTQC with low space overhead [14]. This asymptotic promise led to the development of many families of QLDPC codes with practical parameters [15], notably the Bivariate Bicycle (BB) codes introduced by IBM [16].

To perform computation on encoded quantum information, many schemes and architectures have been proposed for surface codes [10], [17]–[20] and QLDPC codes [14], [21]–[27]. An essential component common to almost all existing architectures is the production of high-fidelity non-Clifford resource states known as *magic states*. These magic states can be consumed through gate teleportation [28] to fault-tolerantly implement non-Clifford logical gates on encoded information. Conceptually, gate teleporting magic states is used in most architectures because non-Clifford gates, without which we cannot perform universal quantum computation, are at this time more costly to implement using other common mechanisms such as transversal gates. Gate teleportation serves as an approach where most of the cost of non-Clifford gates is offloaded to magic state preparation, while the active cost at runtime (namely performing the teleportation) is relatively lower. In most architectures, the total overhead and logical gate speed are often bottlenecked by the costs of the magic state factories.

The most widely applied method of magic state production is called *magic state distillation* (MSD) [29], which consumes many copies of noisy, lower-fidelity magic states to produce fewer copies of higher-fidelity magic states. This procedure is applied to logical qubits, and typically iterated to suppress logical error rate to the scale needed for large scale FTQC ($\leq 10^{-12}$). Conventional MSD factories are therefore very resource-intensive, often becoming the architectural bottleneck. An alternative approach called *magic state cultivation* (MSC) [30] gained recent attention due to its impressive practical performance, achieving a logical error rate of 2×10^{-9} at 10^{-3} physical error rate. MSC injects a physical magic state into a surface code logical qubit, and suppresses infidelity by applying multiple rounds of post-selection and physical non-Clifford gates. Due to its use of exponentially scaling post-selection, MSC does not scale asymptotically and may not suffice for large scale FTQC.

In this work, we introduce new designs of magic state factories that integrate MSC on surface code and MSD in BB codes, achieving low logical error rates while using significantly fewer resources compared to conventional distillation factories. In more detail:

- We present a collection of MSD protocols that run

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00148

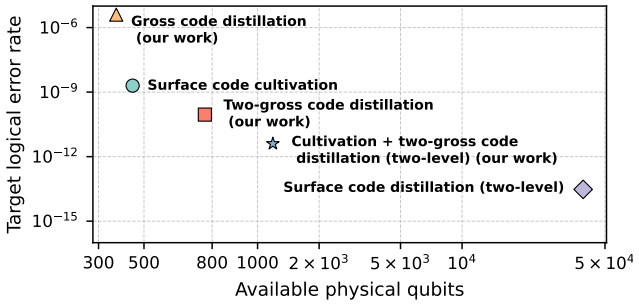

Fig. 1: Magic-state factory design space. Target logical error rate as a function of available physical qubits for surface-code cultivation, BB-code distillation on gross and two-gross codes, and two-level protocols combining cultivation with two-gross or surface-code-only distillation at physical error rate $p_{\text{phys}} = 10^{-3}$. Our BB-based factories achieve lower output error at similar qubit budgets, and the cultivation + two-gross pipeline extends to lower error regimes than cultivation can reach.

entirely within a single BB code block[1], in contrast to conventional factories which run MSD on multiple blocks of QLDPC codes or surface codes. This one block design confers significant savings in the space overhead of magic state factories, reducing the physical qubit count from thousands to hundreds. Additionally, it simplifies the architecture, decreases the number of long range inter-block connections, and reduces decoding complexity.

- We present comprehensive, end-to-end optimizations for existing MSD circuits, including (i) logical qubit mapping to maximize the use of native, low cost logical gates[2] on the BB code, (ii) a masking technique that augments rotations with Z on inactive qubits to turn a set of sparse native logical operation on all logical qubits into a denser set over the active logical qubits, (iii) gate scheduling cast as a Traveling Salesman Problem to minimize compiled circuit depth, and (iv) parallel execution of two MSD protocols on the same BB code block.
  Together, these methods significantly reduce the circuit depth of MSD and improve the factory throughput.
- We introduce a general method to compress MSD circuits to be supported on fewer logical qubits while maintaining the same level of logical error suppression. As examples, we compress the 49-to-1 protocol from 13 to 7 qubits, the 51-to-3CS protocol from 18 to 9 qubits,the 64-to-2CCZ protocol from 17 to 10 qubits, making these circuits implementable within a single BB code block. Moreover, this method is generic to any magic state distillation protocol generated by tri-orthogonal matrices. As near-term quantum computers are space-limited, our method brings large MSD protocols significantly closer to practice.
- We benchmark a collection of magic state factories with varying protocols, codes, and noise levels. Our results quantify the substantial overhead reduction enabled by the proposed optimizations, and, through sensitivity analysis, provide practical guidance for protocol design under different operating regimes, while highlighting the key bottlenecks and improvement opportunities for each design.

As shown in Figure 1, our leading proposal is to perform 15-to-1 MSD on a block of two-gross code (an instance of BB code), using logical magic states supplied by MSC on a surface code. Under 10^{-3} physical error rate, this protocol is estimated to reach logical error rates lower than what is currently achievable through MSC, while using just one block of surface code and one block of two-gross code. Our factories naturally fit into the bicycle architecture [27], a promising near-term FTQC architecture based on the BB codes [16]. Conceptually, the bicycle architecture encodes information in BB codes and performs logical operations using recently-developed code surgery techniques [31]–[34]. These operations consume magic states, which makes magic generation the central component that determines the overall efficiency. Our designs thereby constitute an essential improvement to the bicycle architecture. For general extractor architectures [26], our techniques can also be applied to design other efficient magic state factories.

The rest of this paper is organized as follows. Sec. II reviews the fundamentals of quantum error correction, the BB code family, and magic state distillation. In Sec. III, we present a full stack BB code based magic state distillation factory design. Sec. IV introduces multiple optimization methods to reduce both the depth and the size of the MSD circuit. Sec. V introduces a protocol-level compression method to reduce the logical-qubit footprint of the MSD circuit by recycling inactive logical qubits. In Sec. VI and Sec. VII, we evaluate the space time cost and output logical error rates of our approach and compare them with surface code based magic state distillation and cultivation.

II. Background

A. Quantum Error Correction

Quantum Error Correction (QEC) is crucial for the development of scalable quantum computation [2]. A quantum error correction code has parameters $[[n, k, d]]$, where n is the number of physical qubits in hardware, k is the number of encoded logical qubits which experience suppressed error rates and are used for computation, and the distance d is a measure for the degree of error suppression. The standard surface code has parameters $[[d^2 + (d-1)^2, 1, d]]$ for variable d.

Quantum low-density parity-check (QLDPC) codes, generalized from classical low-density parity-check (LDPC) codes, are particularly attractive for scalable fault-tolerant quantum computing due to their high encoding rates, which reduce the space overhead of FTQC, and sparse check structure, which

[1]More specifically, the MSD circuits are supported within a single BB code block, while the input magic state may be supported externally.

[2]These gates are performed through code surgery with the help of an ancilla system, which we detail in Section II.

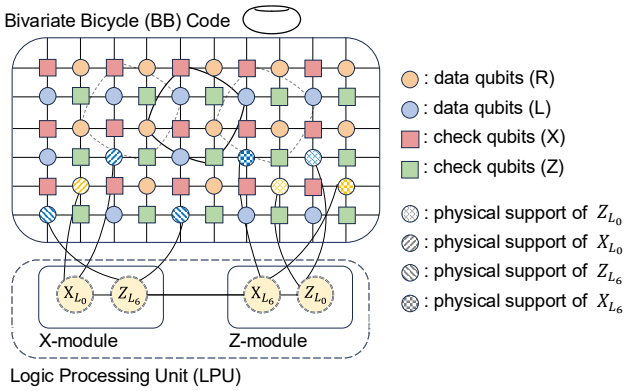

Fig. 2: One module in the bicycle architecture, consisting of a Bivariate Bicycle (BB) code and a Logic Processing Unit (LPU). BB codes can be presented [16] as a lattice of physical qubits on a torus, consisting of X and Z check qubits and L and R data qubits. Connections between qubits are within the lattice and also along certain long-range connections. The LPU, which consists of two modules, is connected to the X and Z operators of the pivot qubit L_0 and its dual L_6. The LPU is capable of measuring any product of operators $X_{L_0}, Z_{L_0}, X_{L_6}$, and Z_{L_6}. See Section II-C for more details.

allows for efficient decoding algorithms and locality in physical implementation. Recent advances in constructing high-rate codes [16], [35]–[37], implementing logical operations [26], [32], [38], [39], and building hardware with long-range connections [40]–[43] have established QLDPC codes as promising candidates for near-term quantum memories and FTQC.

B. Bivariate Bicycle Codes

The Bivariate Bicycle (BB) codes are a family of recently proposed QLDPC codes [16], with the "gross" [[144,12,12]] and "two-gross" [[288,12,18]] codes being particularly attractive due to their high encoding rate and promising hardware realizability. Schemes for implementing logical operations vary significantly across different code families, so we focus our attention on BB codes for this work.

We give a brief mathematical definition of BB codes. Consider an abelian group $\mathcal{M} = \{x^i y^j \mid i \in [0, \ell - 1], j \in [0, m-1]\}$, where indices are taken modulo ℓ and m. This group admits a representation in terms of permutation matrices $\mathbb{F}_2^{\ell m \times \ell m}$. To specify a BB code, we select ℓ, m and elements $A_1, A_2, A_3, B_1, B_2, B_3$ of $\mathcal{M}$. We form the matrices $A = A_1 + A_2 + A_3$ and $B = B_1 + B_2 + B_3$ via the permutation representation, and define parity-check matrices as $H_X = [A \mid B]$ and $H_Z = [B^\top \mid A^\top]$, where $\top$ denotes transposition. The $n = 2\ell m$ columns of these matrices denote physical qubits, and the rows denote supports of X- and Z-type stabilizers for H_X and H_Z respectively. By construction, all checks are supported on six data qubits and all data qubits participate in six checks. This leads to a natural realization on superconducting architectures with $2\ell m$ data qubits and $2\ell m$ check qubits, which are the vertices of a connectivity graph with degree six.

C. Fault-tolerant Architectures Based on QLDPC Codes

Fault-tolerant computation on QLDPC codes can be realized through fault-tolerant measurement of logical Pauli operators assisted with high-fidelity magic states [44]. To implement logical measurements, a popular technique is generalized code surgery [26], [27], [31]–[34], [45], which appends a carefully constructed ancillary system of physical qubits to act as a measurement gadget. Recent developments in surgery have introduced techniques to construct highly flexible ancilla systems, which led to the proposal of the bicycle architecture [27] and its generalization, extractor architectures [26]. In these architectures, logical information is encoded in multiple QLDPC code blocks, each augmented by an ancilla system called an extractor or, for BB codes, a Logic Processing Unit (LPU). These ancilla systems are connected together to enable flexible measurements of many logical operators and, for universal FTQC, are further connected to magic state factories. Our work focuses on the bicycle architecture and its magic state factories.

The bicycle architecture [27] relies on a combination of a highly optimized LPU with fault-tolerantly realizable unitary automorphism gates. The LPU construction exploits symmetries in the Bivariate Bicycle codes to maximize measurement capabilities while minimizing additional physical qubit count. For example, the [[144,12,12]] code admits an LPU with 90 qubits, and the [[288,12,18]] code admits an LPU with 158. We briefly discuss the logical gate set in the bicycle architecture.

1) Automorphism gates: An automorphism gate is a permutation of physical qubits which enacts a `CNOT` circuit on the logical qubits. Within the group of automorphisms, we focus on a 36-element subgroup of *shift automorphisms* that preserve X/Z type. Twelve of these can be implemented fault-tolerantly using the same sparse connectivity as syndrome extraction. These 12 elements form a convenient generating set: any shift automorphism can be synthesized using at most two generators.

A key structural feature of BB codes is their ZX-duality [38]: an order-two permutation of physical qubits followed by a layer of Hadamards maps performs a logical gate. A basis for the twelve logical qubits is chosen so that they divide into two blocks of six, and these two blocks are swapped and Hadamarded by the ZX-duality. Since the shift automorphisms commute with the ZX-duality, we know their logical unitaries take the form $U_{\text{aut}} \otimes U_{\text{aut}}$ on the two logical blocks.

2) Native measurements and rotations: The LPU enables 15 different Pauli measurements supported on two logical qubits, see Figure 2. Composing these measurements with the 36 shift automorphisms gives us $540 = 15 \times 36$ native multi-qubit Pauli measurements on up to 12 logical qubits. Although this is only a small subset of the full 4^{12} Pauli group, they generate all Clifford operations on 11 logical qubits through a measurement-to-rotation compilation. As shown in [31] and Fig. 4(d), using one logical qubit as a *pivot qubit*, we can implement $\exp(i\frac{\pi}{4}P)$ rotations for multi-qubit Pauli P by measuring P on the desired

qubits and X, Y, Z on the pivot qubit. These *native rotations* implemented by *native measurements* can then be composed to generate the Clifford group on 11 logical qubits.

3) Simultaneous measurements: The LPU attaches independently to two logical qubits, the pivot and its dual, and consists of separate X and Z modules. This naturally suggests that certain pairs of Pauli operators may be measured simultaneously. For example, an $X \otimes X$ measurement across the pivot and its dual can be realized by coupling the X-module and Z-module through a transient bridge system. Removing the bridge implements two single-qubit measurements, $X \otimes I$ and $I \otimes X$, in one logical cycle. Analogous constructions exist for $Z \otimes I$ and $I \otimes Z$. We note that simultaneous measurements can reduce circuit depth and amortize LPU cost.

The native gate sets in the bicycle architecture are benchmarked in [27]. Simulation results indicate that automorphisms are substantially less error-prone than LPU measurements, although their cost depends on the number of generators used. Automorphism gates are also much faster than LPU measurements. Our optimization and compilation of MSD protocols therefore aim to minimize the number of LPU measurements.

D. Magic State Distillation

In almost all modern FTQC architectures, logical non-Clifford gates, such as the $T = \mathrm{diag}(1, e^{i\pi/4})$ gate, are implemented through various forms of gate teleportation [28]. These protocols consume magic states, such as $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ for the T gate. Efficient preparation of high-fidelity magic states is therefore crucial for the performance of these architectures. For most quantum codes, preparation of high-fidelity magic states is not possible directly in an error-corrected setting. However, noisy magic states can be prepared outside of the protection of an error-correction code, and then loaded into a logical qubit so further errors do not accumulate. Such noisy state injection produces $|T\rangle$ with error rate p_{inj} that is typically much larger than the target logical error rate p_{L} required for an algorithm. To bridge this gap, one uses *magic state distillation* (MSD): a fault-tolerant protocol that consumes multiple noisy copies of a magic state to produce fewer, higher-fidelity outputs. Intuitively, MSD trades quantity for quality, suppressing errors by leveraging redundancy and special algebraic structure in the underlying code.

Most magic state distillation protocols are constructed from $[[n, k, d]]$ stabilizer codes, which are sometimes called distillation codes. Unlike codes that are used for quantum memory like BB codes, distillation codes feature additional algebraic constraints such as featuring a transversal T gate, allowing them to map n noisy input magic states to k improved output magic states. A single round achieves output error of the form $p_{\mathrm{out}} \approx c\,p_{\mathrm{inj}}^{t} + O(p_{\mathrm{L}})$, where $t = O(d)$ is the degree of error suppression (in many protocols $t = d$), c is a protocol-dependent constant, and p_{L} captures additional faults from running the MSD circuits on logical qubits. A notable $[[15, 1, 3]]$ protocol distills $|T\rangle$ states with $c = 35$ and $t = 3$ [29].

Repeated application of a distillation protocol results in substantial error suppression. Chaining r rounds yields $p_{\mathrm{out}}^{(r)} \sim \tilde{c}\,p_{\mathrm{inj}}^{t^r}$ at the cost of increased qubit and time overhead. For this reason, up to two rounds of distillation suffice for many applications.

III. Overview: Magic State Distillation in BB Architectures

We design magic state distillation factories tailored to the Bivariate Bicycle (BB) architectures of [27], [31]. Our goal is high-fidelity, low-latency $|T\rangle$-state distillation that respects the locality, modular structure, error pathways, and syndrome-extraction capabilities of BB codes. We first show how standard MSD protocols map cleanly onto the BB layout, then present several techniques for implementing the required multi-qubit $\exp(i\frac{\pi}{8}P)$ rotations via injected $|T\rangle$ states. Throughout, we highlight how inter-module connectivity, pivot-qubit access, and measurement fidelity influence factory design.

A. Magic State Distillation with Triorthogonal Matrices

We focus on $|T\rangle$-state distillation following the construction of [46], where each protocol is defined by a *triorthogonal* matrix [47]. Let $G \in \{0,1\}^{m\times n}$ be such a matrix with k rows of odd weight (corresponding to the k output qubits) and $m-k$ rows of even weight (corresponding to ancilla qubits). Each column c specifies a commuting Z-type $\pi/8$ rotation acting on the set of qubits/rows $S_c = \{r : G_{rc} = 1\}$.

Given G, the protocol proceeds as follows:

1) Initialize m logical qubits to $|+\rangle$. In the BB architecture, this is achieved by preparing all physical qubits in $|+\rangle$ and running one syndrome-extraction cycle.
2) For each column c, consume one noisy $|T\rangle$ and implement $e^{i\frac{\pi}{8}P}$. Here $P = Z^{\otimes S_c}$ is an m-qubit Pauli with Z on rows in S_c and identity elsewhere.
3) Measure the $m-k$ parity check qubits in the X basis and postselect on the all-$|+\rangle$ outcome. When postselection succeeds, the first k rows contain the distilled $|T\rangle^{\otimes k}$. In BB codes, these measurements can be implemented in $k+1$ logical steps as follows: for each output magic state, fix a target qubit (on another code block) and measure $Z\otimes Z$ between the magic state and the target qubit; then measure all physical qubits in the X basis. This procedure projects the output magic states onto their destinations.

This formulation uses only m logical qubits, unlike the original method of [47], which prepared an n-qubit stabilizer state, applied T to each qubit, and unencoded via Clifford operations. It preserves the same distillation performance with far fewer qubits and Clifford gates, making it well-suited to BB codes where logical qubit count is tightly constrained.

B. Implementing $\pi/8$ Rotations

Implementing the protocol reduces to realizing $\exp(i\frac{\pi}{8}P)$, where $P = Z^{\otimes S_c}$. We adapt three approaches from [46] to the BB architecture (Figure 4). The key distinction among them is how they handle the necessary conditional Clifford correction:

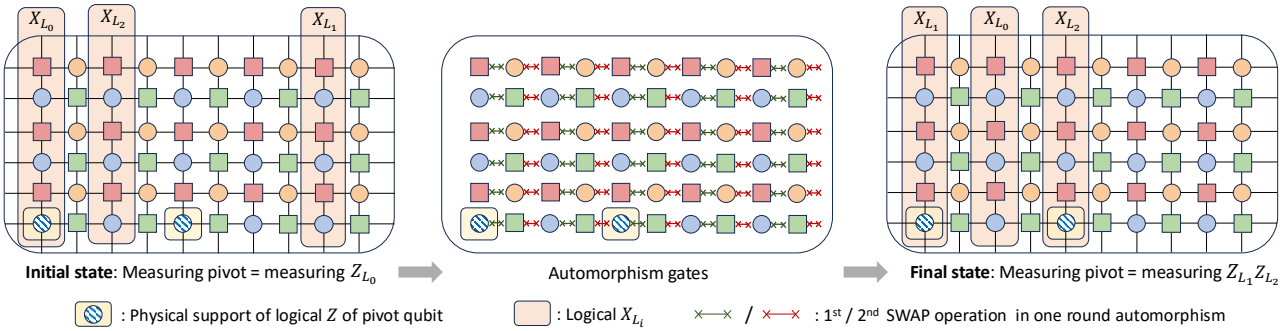

Fig. 3: Fault-tolerant implementation of a shift-automorphism generator and its impact on logical operators. Shift automorphisms permute data qubits via successive swap operations (green, then red) between data and check qubits along edges in the connectivity graph. Logical operators $X_{L_0}, X_{L_1}, X_{L_2}$ supported on shaded regions are permuted so that their overlap with the pivot's Z_{L_0} support changes. After conjugation, multi-qubit Paulis that were not directly accessible through the pivot become measurable via an LPU Z_{L_0} measurement.

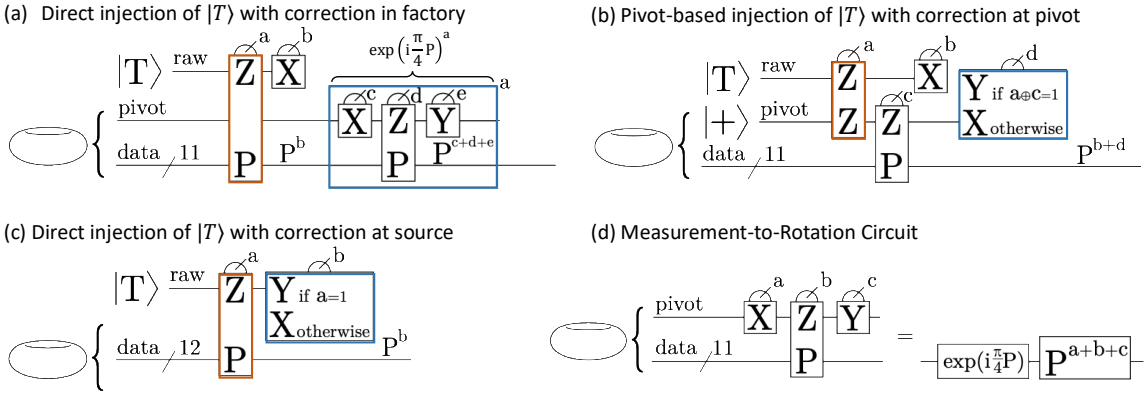

Fig. 4: (a–c) Magic state injection schemes for implementing $\exp(i\frac{\pi}{8}P)$ in a BB architecture. The schemes differ in how the magic state is teleported to the target qubits and how the resulting conditional Clifford correction is handled, leading to different latency and error profiles. Inter-module measurements are shown in orange, conditional Clifford corrections in blue. A P label denotes a Pauli operator, which is cheap to track in fault-tolerant architectures. (d) Measurement-to-rotation circuit implementing $\exp(i\frac{\pi}{4}P)$ using a designated pivot qubit and BB's toric symmetry. We use pivot injection as the default throughout the paper because it achieves a lower error rate, as demonstrated by the detailed benchmarking results in Sections VI-C and VII.

standard injection produces either $\exp(+i\frac{\pi}{8}P)$ or $\exp(-i\frac{\pi}{8}P)$ at random, so a corrective $\exp(i\frac{\pi}{4}P)$ may be required. The three approaches are as follows:

- **Direct injection with factory correction: Fig.4(a).** The input $|T\rangle$ is teleported directly onto all target qubits via an inter-module measurement. Any required correction is implemented explicitly using the measurement-to-rotation circuit (Fig. 4(d)). The injection itself does not require the pivot, but the correction does.
- **Pivot-based injection with pivot correction: Fig.4(b).** The $|T\rangle$ state is first teleported onto the pivot qubit, then onto the target qubits using only in-module measurements. This confines the noisy injection step to a single qubit and avoids spreading error across the data block. The correction is absorbed into a conditional X or Y measurement on the pivot.
- **Direct injection with source correction: Fig.4(c).** If the module supplying the $|T\rangle$ states supports direct, high-fidelity Y measurements, the correction can be applied by the source qubit itself. In this case, the pivot is not involved at all, and no additional correction step is required. Whether this is viable depends on the native measurement bases of the $|T\rangle$-state source.

These three strategies span different hardware assumptions and error models. In Section VI-C, we quantify how their measurement counts, routing demands, and error locations translate into overall factory throughput and logical error rates for realistic BB-code parameters. In Section VII, we present detailed benchmarks showing how different injection schemes can be selected adaptively under different hardware assumptions, and we explore the tradeoff between spacetime volume and output error rate across these schemes.

IV. Implementation and Optimizations

In this section, we present techniques that improve the efficiency and reliability of magic state distillation within the bicycle architectures. These optimizations address bottlenecks from restricted native measurements, limited logical qubits, and architectural error sources. Together, they define a practical workflow for compiling distillation protocols into fault-tolerant BB-code factories with minimal overhead.

A. Logical Qubit Mapping: Maximizing Native Coverage

Because the native measurement set is limited, not every Pauli P required by the protocol can be realized by a single LPU measurement, even after conjugation. However, the protocol uses only a fixed set of logical qubits, which we are free to place within the BB code.

We therefore treat logical-qubit placement as an optimization problem. Given a BB code with k data qubits and an m-qubit distillation protocol, we choose an m-element subset $S \subseteq [k]$ and assign protocol qubits to S so that the number of native Pauli rotations is maximized. For the small protocol sizes of interest, we can brute-force over S; ties are broken by minimizing routing distance to the pivot.

For example, in the 15-to-1 protocol on the gross code, choosing 5 of the 6 qubits in a logical block yields native realizations for most of the required $\pi/8$ rotations, with the remainder implemented either by Clifford conjugation [31] or by masking (Section IV-B). This mapping step is lightweight but important: improving native coverage directly reduces factory latency and logical error.

B. Masking: Enabling More Native Measurements

When $m < k$, unused logical qubits can be repurposed to expand the effective native measurement set. Suppose a required rotation $\exp(i\frac{\pi}{8}P)$ is non-native, but there exists a native Pauli Q that matches P on the m active qubits and differs only on a subset of idle qubits. For instance, if those idle qubits are initialized to $|0\rangle$, then applying Z on them leaves the state invariant, since $Z|0\rangle = |0\rangle$. We can therefore replace P by $Q = P \cdot \prod_{j\in\mathcal{M}} Z_j$, where $\mathcal{M}$ is a set of masked qubits chosen so that Q is native.

This *masking* operation is purely logical and adds no depth. It increases the fraction of rotations that can be executed as single native measurements.

In the 15-to-1 protocol, masking fully nativizes all 15 rotations: the four previously non-native Paulis become native when augmented with Z factors on masked qubits. As illustrated in Figure 5, masking allows each rotation to be implemented using a single automorphism sequence and one LPU measurement (up to tracked byproduct Paulis), eliminating the need for additional Clifford conjugation.

C. Gate Scheduling: Reducing Automorphism Rounds

As above, a measurement of a logical Pauli P is implemented by conjugating an LPU-native measurement with one or more automorphism gates. Different automorphisms incur different

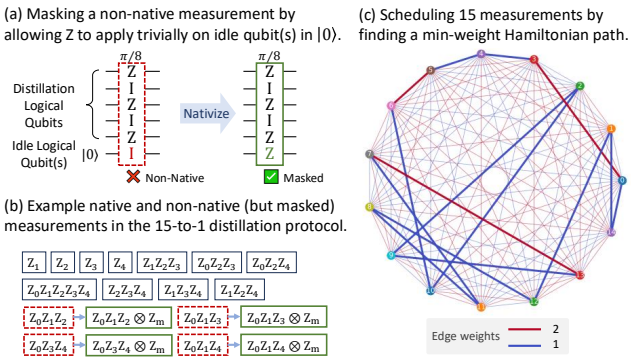

Fig. 5: (a) Masking technique to nativize a Pauli measurement by allowing Z to act on an idle logical qubit initialized to $|0\rangle$ within the BB code. (b) Native and non-native measurements in the 15-to-1 distillation circuit, which becomes fully nativized after masking. (c) Scheduling the 15-to-1 rotations in an order that minimizes automorphism rounds between successive measurements.

costs, typically corresponding to one or two automorphism-generator applications. In injection schemes that do not require intermediate pivot measurements between successive $\exp(i\frac{\pi}{8}P)$ gates, such as direct injections (Figure 4), we can reduce total automorphism cost by optimizing the order of Pauli rotations.

All $\exp(i\frac{\pi}{8}P)$ gates in a triorthogonal distillation protocol commute, so we are free to reorder them without changing the logical channel. The scheduling problem thus reduces to finding an execution order that minimizes the cumulative automorphism overhead needed to retarget the LPU between consecutive measurements.

We model this as a graph problem. Each distinct Pauli label P in the protocol is represented as a node v in a directed graph $G = (V, E)$. For any ordered pair (u, v), we define the edge weight $w(u, v)$ as the cost of transforming the measurement configuration for u into that for v using automorphisms. This cost can be defined in terms of the number of automorphism rounds, latency, or any hardware-informed metric.

Any ordering of the rotations corresponds to a permutation σ of the nodes in V, with total routing cost

$$C(\sigma) = \sum_{i=1}^{|V|-1} w\big(v_{\sigma(i)}, v_{\sigma(i+1)}\big).$$

Minimizing $C(\sigma)$ is equivalent to finding a minimum-cost Hamiltonian path from $v_{\sigma(1)}$ to $v_{\sigma(|V|)}$, that is, a Traveling Salesman Problem (TSP) instance with fixed endpoints.

The TSP formulation changes only the measurement order, not the gates themselves or their angles. Although TSP is NP-hard in general, our instances are small; for example, the 15-to-1 protocol has only fifteen distinct $\exp(i\frac{\pi}{8}P)$ rotations. For such sizes, standard heuristics such as nearest-neighbor initialization with 2-opt or 3-opt refinements, or a warm-started mixed-integer linear program, quickly find near-optimal or optimal routes. The automorphism cost matrix can be precomputed

once per BB-code instance and reused across factory cycles, so the marginal scheduling overhead is negligible.

D. Improving Throughput: Multi-Track Distillations

When the triorthogonal matrix G has small row count m, the BB architecture can host multiple protocol instances in parallel on a single code block. For the 15-to-1 and 8-to-CCZ protocol [29], [46], $m = 5$ ($m = 4$ for 8-to-CCZ) fits comfortably into each six-qubit logical block of the 12-qubit Gross and two-Gross codes. This enables a natural *dual-track* mode: run two copies of the protocol simultaneously on the two ZX-dual blocks, effectively doubling factory throughput without adding code patches.

As discussed in Section II-C3, qubits L_0 and L_6 form a dual pair under ZX-duality, and the LPU is attached to both. The LPU also decomposes into distinct X and Z modules. When both modules operate in the same basis, the architecture supports simultaneous X or Z measurements on L_0 and L_6. Because the automorphism group acts identically on the two six-qubit blocks, this parallelism extends to more general Pauli measurements, as long as the logical Paulis on the two blocks coincide and are purely X or Z type.

These properties align well with direct-injection schemes (Figure 4), see Fig. 6. For most rotation steps, we can schedule paired measurements on the two blocks with identical logical labels so that one sequence of automorphisms followed by a simultaneous LPU measurement implements both rotations. This yields a near factor-of-two throughput improvement for the same LPU footprint.

The main exception occurs at steps that require Y-basis measurements on the pivot. A Y measurement occupies both the X and Z modules, so the two protocol copies must serialize at those points. In pivot-based injection, the need for a pivot Y measurement is tied to whether a correction is required, which happens with probability $3/4$. In these cases, multi-track execution does not reach a strict factor-of-two speedup, but still provides a significant throughput gain, especially when the protocol is dominated by X and Z rotations and when direct injection reduces pivot usage.

V. Protocol-level Compression of Distillation Footprint: Recycling Logical Qubits

The limited number of logical qubits in BB codes makes it challenging to host large distillation protocols within a single patch. Instead, we reduce the protocol's logical-qubit footprint by exploiting structure in its triorthogonal matrix. This optimization does not rely on implementation-specific details of the distillation circuits, and therefore applies to any distillation protocol generated from triorthogonal matrices, including protocols for states such as $|T\rangle$, $|CS\rangle$, and $|CCZ\rangle$.

We introduce a *protocol compression* technique that lowers the peak number of simultaneously active logical qubits by recycling qubits whose rows become idle. In many triorthogonal constructions, such as the 49-to-1 code of Bravyi and Haah [47], the matrix contains large all-zero subblocks that indicate opportunities to delay initialization or advance measurement.

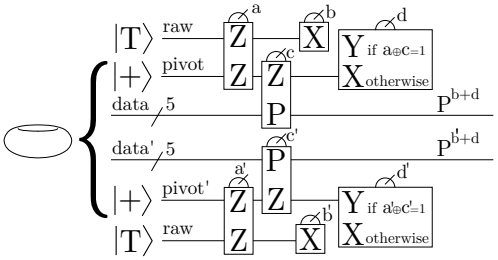

Fig. 6: Simultaneous realization of two pivot-based injections on blocks L_0 to L_5 and L_6 to L_{11}. Logical Paulis on the two blocks are chosen to be identical and of pure X or Z type, which is essential for parallelization in *dual-track* distillation. Only pivot Y measurements, which require both LPU modules, must be serialized. Similar patterns apply to the other injection schemes in Figure 4.

Let $G \in \{0,1\}^{m\times n}$ be a binary triorthogonal matrix. The first k rows have odd Hamming weight and encode the k output logical qubits; the remaining $m-k$ rows have even weight. For each row i, let f_i denote the index of its first 1 (or $+\infty$ if the row is all zeros) and ℓ_i the index of its last 1 (or $-\infty$ if all zeros).

Even rows support stabilizer checks and can both start later and end earlier: if the leftmost nonzero entry is at column f_i, the row need not be initialized before column f_i, and if the rightmost nonzero entry is at ℓ_i, the row can be measured and freed after column ℓ_i. Odd rows, in contrast, encode outputs and cannot be freed once initialized; even if an odd row has trailing zeros, we treat it as active on all columns $j \geq f_i$.

We say row i is *working* on column j if $j \geq f_i$ and either (i) $j \leq \ell_i$ and the row is even, or (ii) the row is odd. Let $W(j)$ denote the set of working rows at column j. The required number of simultaneous logical qubits is

$$\mathcal{C}(G) = \max_{j\in[n]} |W(j)|.$$

Our goal is to transform G into an equivalent triorthogonal matrix G' with the same distillation properties but a smaller peak footprint $\mathcal{C}(G')$.

We allow three classes of transformations that preserve triorthogonality and the encoded protocol:

- Column permutations, which reorder the commuting $\pi/8$ rotations.
- Row permutations within blocks, which reorder odd rows among themselves and even rows among themselves.
- Row additions over $\mathbb{F}_2$, which add one row to another while maintaining triorthogonality and logical content.

By applying these operations, we reshape G so that many even rows share a right-aligned all-zero submatrix (early measurement) and many rows share a left-aligned all-zero submatrix (delayed initialization). In terms of the intervals $[f_i, \ell_i]$, these transformations shorten active windows and reduce the maximum overlap $\max_j |W(j)|$.

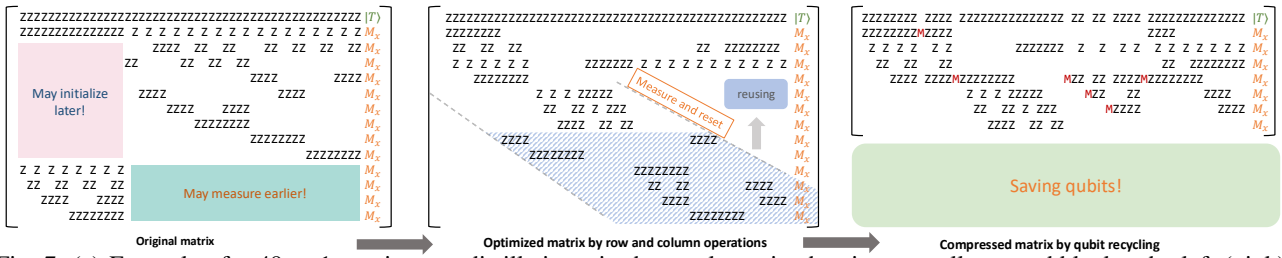

Fig. 7: (a) Example of a 49-to-1 magic state distillation triorthogonal matrix showing two all-zero subblocks: the left (pink) region corresponds to rows that can be initialized later, and the right (green) region to even rows that can be measured earlier. (b) After row and column operations that preserve triorthogonality, logical qubits freed by early-measured rows are recycled to support later-initialized rows, yielding a compressed matrix with fewer simultaneously active rows. (c) The optimized matrix achieves substantial logical-qubit savings while preserving protocol correctness.

Figure 7 illustrates this process for a 49-to-1 protocol. The original matrix has prominent left- and right-aligned all-zero regions. After suitable row additions and permutations, qubits freed by early-measured rows are recycled to support later-initialized rows. The resulting compressed matrix has a significantly reduced peak logical-qubit footprint while preserving triorthogonality and output error suppression.

Finding the globally optimal compression is computationally hard. Even in the simplified case where $k = 0$ and each even row has Hamming weight two, minimizing $\mathcal{C}(G)$ reduces to the NP-hard cutwidth problem. For realistic protocols with n in the tens or higher, we therefore rely on heuristics. In our experiments, simple greedy schemes that cluster row starts and ends, combined with targeted row additions, already yield substantial qubit savings and allow otherwise infeasible protocols to fit within a single BB patch.

VI. Evaluation Methodology

A. Baselines

We compare our proposed distillation factories (GROSS and TWO-GROSS) against two state-of-the-art magic-state factory baselines: a surface-code distillation baseline follows the lattice-surgery factories of Litinski [46], and a cultivation baseline from Gidney's grafted surface code magic state cultivation [30]. Factories are evaluated under two different physical error rates p_{phys}, with various input magic state error rates p_{in} and output magic state error rates p_{out}. Each factory is characterized by its physical-qubit footprint, the number of logical timesteps τ_i per batch, and the resulting space-time volume (qubits $\times$ timesteps). These are the quantities reported and compared in our results.

Distillation baselines are labeled as $(\text{Protocol})_{\text{Code}}$, and, when the protocol is implemented on the surface code, we further annotate a triple (d_X, d_Z, d_m) that specifies the code distances used for the data blocks in [46]. Cultivation baselines are written as $(\text{Cultivation})_{\text{SC}\to d}$, where d is the distance of Gidney's grafted surface-code patch.

B. Factory Usage Modes

Magic state distillation protocols require a source of raw magic states as input. Our protocols then operate on magic states loaded into logical qubits of a bivariate bicycle code. We consider two settings in which our methods can be applied:

Two-round distillation. The bicycle architecture [27] details a promising approach for obtaining magic states by connecting magic state cultivation protocols [30] to a BB memory via a surgery ancilla system called *adapter* [34]. Magic state cultivation is a compressed hardware-native protocol that can achieve error rates as low as 10^{-9} to 10^{-11} depending on the details of the construction. Novel designs of magic state cultivation are still emerging [48] and may need to be tailored to hardware limitations, but the adapter construction is flexible enough such that any such proposal could be integrated into a bicycle architecture. In this setting, cultivation would act as a first-round protocol, with our distillation protocols implemented in BB memory acting as a second-round protocol achieving suppressed error rates (e.g., $35 \cdot (10^{-9})^3 = 3.5 \cdot 10^{-26}$ with a 10^{-9} cultivated state fed into a $[[15, 1, 3]]$ protocol). We also consider more conventional factory designs, where the first and second round protocols are both distillation. In our later results, such configurations are written as a combination of the two round protocols to make the first- and second-round costs explicit.

One-round distillation. While magic state cultivation is an optimized, high-performing protocol, we also consider other first-round protocols for adaptability to different architectural settings. An emerging line of work [39], [49], [50] is considering lower-overhead methods that instead inject low-quality magic states (physical error rates 10^{-2} to 10^{-3}) into the memory directly, in which case our methods would act as a first round of error suppression. These could then be used for applications of early fault-tolerant scale (e.g., $35 \cdot (10^{-3})^3 = 3.5 \cdot 10^{-8}$ with a 10^{-3} raw state in a $[[15, 1, 3]]$ protocol).

C. Noise Model and Error Analysis

A crucial part of magic-state factory design is understanding how each error source affects the final output error rate and the discard (post-selection) rate. Here, we present a detailed noise model and error analysis for our implementation of the distillation circuit in the BB architecture. We explicitly

model logical errors, including imperfect logical operation and imperfect measurement outcomes. For both the gross code and the two-gross code, we characterize the combined impact on the delivered state's fidelity and acceptance probability for the following noise resources:

1) **Magic state input error** p_{in}. This error arises from imperfect preparation of input T magic states. We model it as a depolarizing channel applied to the ideal magic state $|m\rangle$. The resulting error should be suppressed polynomially by the distillation protocol.
2) **Automorphism gate error** p_{auto}. Automorphism errors are introduced by physical `CNOT` gates (each with physical error rate p_{phys}) applied to data and ancilla qubits. We model these gate errors as a uniform depolarizing channel acting on each logical qubit.
3) **Inter-module measurement error** p_{inter}. This error is introduced by the adapter that connects the distillation BB code patch with the magic state input. In this paper, we assume such error is concentrated on the two qubits measured, modeled as two-qubit depolarizing logical errors. In the bypassing pivot injection scheme in Figure 4(a)(c), the magic state is injected directly into logical qubits of the distillation circuit, so the logical error channel appears in the injection gadget as a multi-qubit depolarizing channel with inter-module logical error rate. While in the pivot injection scheme in Figure 4(b), Pauli errors introduced by the inter-module measurement are modeled as a two-qubit depolarizing channel acting on the source and pivot qubits, and are then propagated to the injected T magic state at the pivot, though at the cost of additional measurement operations. Below we detail how these errors propagate from the magic state resource with a concrete example.
4) **In-module measurement error** p_{intra}. This error is introduced by the LPU, which is used for logical operations including magic state injection, correction, and final post selection. The total error rate contains two components: measurement error and logical qubit error. The former correspond to flipping of measurement results, and the latter correspond to depolarizing logical errors. We define λ as the ratio between measurement error and the total in-module error rate, $\lambda = \frac{p_{\text{meas}}}{p_{\text{intra}}}$.

Not all logical errors harm the output magic state [46]. Figure 8 gives examples of how faults from logical operations propagate through the distillation circuit and contribute to the output error. In Fig. 8(a1), a Pauli Z logical error on qubits 2 to 5 commutes through all rotations and is detected by the final detector. In contrast, a Pauli X error in Fig. 8(a2) flips every measurement outcome it meets and is invisible to the final detector. Even in this case, it does not necessarily cause an output error, since at least three flipped rotations are required to induce a logical fault; the combination of the X-error location and the gate schedule is therefore crucial.

In Fig. 8, logical measurement errors are split into the two models introduced above. Panel (b1) models a two-qubit

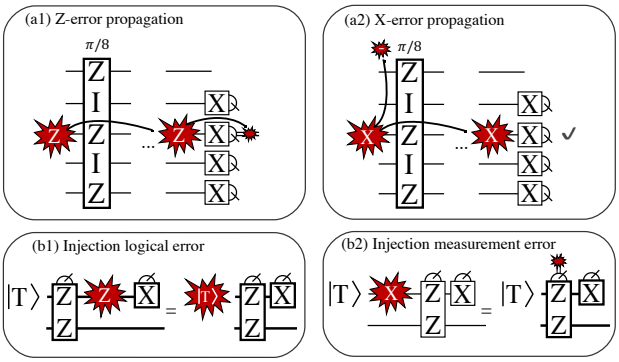

Fig. 8: Illustration of how different errors are handled in our simulations. (a1) A Pauli-Z logical error leaves rotations unaffected but may flip final parity checks. (a2) A Pauli-X error flips the sign of rotations, but leaves final measurements unaffected. (b1) A faulty in-module/inter-module measurement introduces logical depolarizing errors to qubits, and (b2) a fault measurement outcome can be interpreted as a faultier input magic state.

depolarizing channel acting on the measured qubits for inter-module measurements and all qubit depolarizing channel for in-module measurements. Specifically, the error on the magic state qubit is less harmful, as any Z component is equivalent to a Z preparation error that introduces an extra $\pi/2$ rotation, while an X component can be absorbed into the final X-basis measurement. Panel (b2) illustrates a pure measurement-outcome flip, which is equivalent to inserting (or omitting) a $\pi/4$ rotation (or an input X error on the magic state) and is detectable by the distillation protocol. Finally, errors in the final X-basis measurements at the end are generally less harmful as well: a false positive only increases the discard rate, while a false negative must combine with a preexisting logical error and is therefore a second-order contributor to the final output error rate.

Hence, we report the output error rate by two methods: (i) a union bound calculation, which counts any failure from any logical operation toward the output infidelity; and (ii) a density matrix simulation that models each logical qubit as a single qubit with the prescribed logical level noise rates. In the simulation, parameter λ sets the mix of logical measurement errors (measurement flips versus depolarizing memory faults). Both methods employ error rates from previous simulation results in Table I of [27]. Notably, due to the highly non-Clifford nature of the distillation circuits considered here, end-to-end physical-level simulation is computationally impractical in the regime of interest. Exact simulation of generic noisy non-Clifford circuits typically requires dense density-matrix methods rather than efficient stabilizer-based techniques, and is therefore in practice restricted to only tens of qubits, often on the order of $\sim$ 20 qubits. Moreover, a full physical-level treatment would need to incorporate repeated syndrome extraction, decoding, and memory noise over time in order to

Logical Operation	Code Type	Timesteps τ_i	Logical Error Rate P	
			$p_{\text{phys}} = 10^{-3}$	$p_{\text{phys}} = 10^{-4}$
Shift automorphism	Gross	14	$10^{-6.4}$	$10^{-12.2}$
p_{auto}	Two-gross	14	$10^{-14.5}$	10^{-37}
In-module meas.	Gross	120	$10^{-5.0}$	$10^{-9.0}$
p_{intra}	Two-gross	216	10^{-11}	10^{-20}
Inter-module meas.	Gross	120	$10^{-2.7}$	$10^{-7.3}$
p_{inter}	Two-gross	216	10^{-9}	10^{-18}

TABLE I: Timestep and logical error rate of BB code's logical operations from [27]. The tabulated values are identical to those employed in our simulations.

Optimization pass	Runtime
Logical qubit mapping & masking	$\sim 10\,\text{min}$
Automorphism TSP	$< 3\,\text{s}$
Protocol compressor	$< 5\,\text{s}$

TABLE II: Classical Compilation Overhead of the Optimization Methods in Sec. IV and V for two-gross code and 49-to-1 protocol.

determine the effective logical error rates of the BB operations. Our two-level methodology instead uses prior physical-level studies from [27] determine these hardware-informed logical error rates, and then evaluates the performance of the full distillation circuit at the logical level under the resulting noise model.

D. Experimental Setup and Classical Compilation Overhead

All simulations were conducted on a Macbook Pro with a 10 core CPU and 32 GB of RAM. We also report the classical running time of each optimization in Table II, using the example of the largest protocol 49-to-1 and largest code two-gross.

VII. Results

A. Resource Estimation and Comparison

Table III summarizes the resource requirements of our magic-state factories across multiple distillation protocols, where output error rates are obtained from our logical-level simulations in Section VI with logical error rates from Table I.

Unless explicitly stated otherwise, all numbers in Table III (Table II) and in the corresponding comparison plots use pivot injection in Fig. 4(b). The impact of injection choice is isolated in Table IV, where the direct-injection family in Fig. 4(a),(c) is compared against pivot injection under the same factory settings.

At a physical error rate of $p_{\text{phys}} = 10^{-3}$, gross-code distillation used as a one-round factory provides substantial qubit savings compared to surface-code baselines, at the cost of larger τ_i. For example, $(15\text{-to-}1)_{\text{Gross}}$ uses 378 qubits versus 4620 for $(15\text{-to-}1)_{\text{SC(17,7,7)}}$, while requiring larger depth/volume ($6122,\ 2.3 \times 10^6$ vs $256,\ 1.2 \times 10^6$). Because native-measurement logical error in the gross code is relatively high, gross code is better matched to quadratic-suppression protocols (rows $(8\text{-to-CCZ})^{\otimes 2}_{\text{Gross}}$, $(20\text{-to-}4)_{\text{Gross}}$), whereas stronger suppression is obtained in higher-distance two-gross rows ($(15\text{-to-}1)_{\text{Two-gross}}$, $(49\text{-to-}1)_{\text{Two-gross}}$).

While the cultivation scheme still has the lowest space-time overhead for target output error rates above about 10^{-9}, two-gross factories can reach significantly lower output error: $(49\text{-to-}1)_{\text{Two-gross}}$ achieves 10^{-11}-level output for 734 qubits at $p_{\text{phys}} = 10^{-3}$, and $\leq 10^{-17}$ at $p_{\text{phys}} = 10^{-4}$. Its second-round footprint is over $20\times$ smaller than the second-round surface-code stage in $(15\text{-to-}1)_{\text{SC(11,5,5)}} + (15\text{-to-}1)_{\text{SC(25,11,11)}}$ (734 vs 18280).

We also benchmark several two-round factories, analogous to the single-round case. At $p_{\text{phys}} = 10^{-3}$, $(\text{Cultivation})_{\text{SC}} + (15\text{-to-}1)_{\text{Two-gross}}$ reaches 10^{-12}-level output error with comparable second-round volume to $(15\text{-to-}1)_{\text{SC(11,5,5)}} + (15\text{-to-}1)_{\text{SC(25,11,11)}}$ (8.1×10^6 vs 9.1×10^6). At $p_{\text{phys}} = 10^{-4}$, the same hybrid reaches $\leq 10^{-17}$, compared with 7.2×10^{-14} for $(15\text{-to-}1)_{\text{SC(7,3,3)}} + (8\text{-to-CCZ})_{\text{SC(15,7,9)}}$, but with a larger second-round volume (8.1×10^6 vs 2.0×10^6). These row-to-row comparisons show that BB-based second rounds are particularly effective when very low output error is the primary target; because prior surface-code references provide only limited directly comparable baselines (without a fully matched all-surface cultivation+distillation hybrid), we report second-round volume for two-round rows.

Taken together, our results indicate that MSD on BB codes is a compelling building block for multi-round magic state factories. We further note that the simulated error rates of the LPU measurements from [27] are likely to improve in the future, which means the performance of our proposed factories will improve as well.

B. Expanding Native Measurements by Logical Mapping and Masking

As discussed in Sec. IV-A and IV-B, logical qubit mapping and masking increase native measurements in the gross and two-gross codes. Figure 9(a) compares mapping orders and shows that our search-based mapper outperforms brute-force and randomized baselines, making the 15-to-1 and 8-to-CCZ protocols fully native in both codes. Figure 9(b) shows that the native-measurement ratio grows with the number of masked qubits. We define this ratio as the number of native measurements divided by the number of candidate operators, evaluated for two operator sets: the full Pauli set $\{I, X, Y, Z\}$ and an I/Z-only set. Masking substantially increases the share of I/Z-only native measurements required by the magic-state distillation circuits, and the same technique also benefits other quantum circuits executed on the BB architecture when blocks are not fully occupied. The resulting increase in native measurements reduces the synthesis cost of arbitrary Clifford operations and, in turn, lowers the overall circuit depth.

C. Reducing Automorphism Rounds Using TSP optimization

We schedule the commuting $\pi/8$ rotations by solving a TSP-style ordering problem over Pauli measurements in Sec. IV-C. Figure 5(c) illustrates the key effect: instead of following the protocol's original column order, we cluster successive rotations that require similar automorphism actions, which reduces retargeting overhead between measurements. This

One-Round Factories	p_{phys}	p_{in}	Physical Qubits	Timesteps τ_i	Space-time Volume	Union Bound	Simulated	
							p_{out} ($\lambda = 0.9$)	p_{out} ($\lambda = 0.5$)
(15-to-1)$_{\text{SC(17,7,7)}}$ [46]	10^{-3}	10^{-3}	4620	256	1.2×10^6	N/A	4.5×10^{-8}	
(Cultivation)$_{\text{SC}\to\text{d=3}}$ [27], [30]	10^{-3}	10^{-3}	454	351	1.6×10^5	N/A	3×10^{-6}	
(Cultivation)$_{\text{SC}\to\text{d=5}}$ [27], [30]	10^{-3}	10^{-3}	463	2167	1.0×10^6	N/A	2×10^{-9}	
(8-to-CCZ)$^{\otimes 2}_{\text{Gross}}$	10^{-3}	10^{-3}	378	1570	5.9×10^5	3.2×10^{-4}	8.8×10^{-5}	9.8×10^{-5}
(15-to-1)$_{\text{Gross}}$	10^{-3}	10^{-3}	378	6122	2.3×10^6	5.0×10^{-4}	1.3×10^{-6}	4.6×10^{-6}
(20-to-4)$_{\text{Gross}}$	10^{-3}	10^{-3}	378	3088	1.2×10^6	9.8×10^{-4}	4.3×10^{-5}	5.2×10^{-5}
(15-to-1)$_{\text{Two-gross}}$	10^{-3}	10^{-3}	734	11249	8.3×10^6	1.1×10^{-8}	1.0×10^{-8}	1.0×10^{-8}
(49-to-1)$_{\text{Two-gross}}$	10^{-3}	10^{-3}	734	70748	5.1×10^7	3.1×10^{-9}	2.0×10^{-11}	9.7×10^{-11}
(15-to-1)$_{\text{SC(11,5,5)}}$ [46]	10^{-4}	10^{-4}	2070	180	3.7×10^5	N/A	1.9×10^{-11}	
(15-to-1)$_{\text{Gross}}$	10^{-4}	10^{-4}	378	5999	2.3×10^6	5.1×10^{-8}	9.4×10^{-11}	4.2×10^{-10}
(15-to-1)$_{\text{Two-gross}}$	10^{-4}	10^{-4}	734	11090	8.1×10^6	1.0×10^{-11}	1.0×10^{-11}	1.0×10^{-11}
(49-to-1)$_{\text{Two-gross}}$	10^{-4}	10^{-4}	734	68595	5.0×10^7	1.2×10^{-17}	$\leq 10^{-17}$	$\leq 10^{-17}$

Two-Round Factories	p_{phys}	p_{in}		Physical Qubits 1st+2nd Round	Timesteps τ_i	2nd Round Volume	Union Bound	Simulated	
		1st	2nd					p_{out} ($\lambda = 0.9$)	p_{out} ($\lambda = 0.5$)
(15-to-1)$_{\text{SC(11,5,5)}}$ + (15-to-1)$_{\text{SC(25,11,11)}}$ [46]	10^{-3}	10^{-3}	N/A	$12420 + 18280$	495	9.1×10^6	N/A	2.7×10^{-12}	
(Cultivation)$_{\text{SC}}$ [27], [30]+(15-to-1)$_{\text{Two-gross}}$	10^{-3}	10^{-3}	10^{-6}	$454 + 734$	11080	8.1×10^6	4.0×10^{-10}	8.2×10^{-13}	4.1×10^{-12}
(15-to-1)$_{\text{SC(7,3,3)}}$ + (8-to-CCZ)$_{\text{SC(15,7,9)}}$ [46]	10^{-4}	10^{-4}	$10^{-7.36}$	$3240 + 9160$	217	2.0×10^6	N/A	7.2×10^{-14}	
(15-to-1)$_{\text{SC(7,3,3)}}$ + (20-to-4)$_{\text{SC(13,5,7)}}$ [46]	10^{-4}	10^{-4}	$10^{-7.36}$	$3240 + 8340$	420	3.5×10^6	N/A	1.4×10^{-12}	
(15-to-1)$_{\text{SC(7,3,3)}}$ [46] +(8-to-CCZ)$^{\otimes 2}_{\text{Two-gross}}$	10^{-4}	10^{-4}	$10^{-7.36}$	$1620 + 734$	2893	2.1×10^6	2.4×10^{-14}	2.4×10^{-14}	2.4×10^{-14}
(15-to-1)$_{\text{SC(7,3,3)}}$ [46] +(20-to-4)$_{\text{Two-gross}}$	10^{-4}	10^{-4}	$10^{-7.36}$	$810 + 734$	5235	3.8×10^6	1.9×10^{-14}	1.1×10^{-14}	1.1×10^{-14}
(Cultivation)$_{\text{SC}}$ [27], [30] +(15-to-1)$_{\text{Two-gross}}$	10^{-4}	10^{-4}	10^{-6}	$454 + 734$	11080	8.1×10^6	1.0×10^{-17}	$\leq 10^{-17}$	$\leq 10^{-17}$

TABLE III: Resource comparison between different magic state factories across different distillation protocols using pivot-injection. For each factory we list the physical error rate p_{phys}, input magic-state error p_{in}, physical qubit footprint, timestep depth τ_i (discard rate counted), the resulting space-time volume (qubits $\times$ timesteps), and the output error p_{out} obtained from the analytic model and from density-matrix simulation; see Sec. VI-A for definitions and the baseline configurations. Two-round factories are written as "first-round + second-round" and the reported resources correspond to the second round. Entries of the form $(\cdot)^{\otimes 2}$ denote dual-track factories, as described in Sec. IV-D. All benchmarks in this table adopt pivot injection in Figure 4(b) for better inter-module error suppression. Across the listed baselines, gross and two-gross factories often reduce physical-qubit footprint while trading off depth and, in some settings, output error.

optimization lowers both latency and accumulated logical-operation error because each eliminated automorphism round removes additional gates and measurement opportunities for faults. All timestep counts τ_i reported in Tables III and IV include this optimized scheduling.

D. Benchmarking Distillation Protocol Compressor

As discussed in Sec. V, our qubit-recycling distillation-protocol compressor reduces the number of simultaneously active logical qubits without increasing the total number of measurements. Figure 9(b) shows its impact on the logical-qubit footprint of large protocols. In particular, we compress the 49-to-1 protocol from 13 to 7 logical qubits, the 51-to-3CS protocol from 18 to 9 logical qubits, and the 64-to-2CCZ protocol from 17 to 10 logical qubits. These reductions are critical and allow both protocols to fit within a single gross or two-gross factory, whose maximum capacity is 11 logical qubits under our pivot-injection schemes.

E. Identifying Dominant Error Sources

We decompose the output magic-state error of each simulated factory into contributions from three noise sources: (i) errors from automorphism gates, (ii) errors from in-module rotations, and (iii) errors in the injected magic states and inter-module measurements at the universal adapter, with Figure 10 summarizing the dominant source for each factory. Most factories fall into one of two regimes: *operation-limited*, where errors from in-module rotations dominate, and *source-limited*, where imperfect input magic states and inter-module injection errors dominate. A practical design target is an operating point where source and operation errors contribute comparably to the final error; for example, at a physical error rate of $p_{\text{phys}} = 10^{-3}$, the logical error of the gross code is too large to support the 15-to-1 protocol reliably, whereas the two-gross code is better matched to larger protocols such as 49-to-1. As logical error rates in the bicycle architecture improve with advances in LPU design and decoder optimization, this decomposition provides a compact indicator for tuning factory configurations to a given hardware error rate.

F. Comparison Between Different Injection Schemes

Table IV highlights a clear throughput-fidelity tradeoff between the two injection families. Direct injection is consistently faster because it avoids additional pivot measurements during T-state injection, but it inherits more logical memory errors from inter-module noise. Pivot injection adds logical measurements and therefore larger τ_i, yet confines inter-module noise to the source-pivot interface and yields better p_{out} when inter-module measurements dominate.

This effect is strongest for factories composed entirely of native measurements. For example, the 15-to-1 gross-code factory at $p_{\text{phys}} = 10^{-3}$ improves from 8.2×10^{-4} with direct injection to 4.6×10^{-6} with pivot injection. For protocols requiring non-native rotations, such as 49-to-1, the depth advantage of direct injection is less consequential. In some

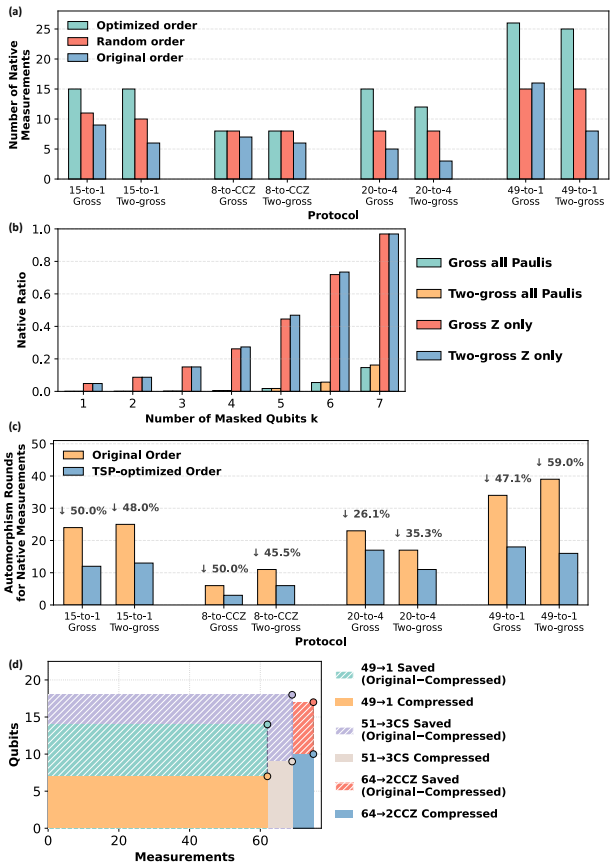

Fig. 9: (a) Increase in native measurements from co-optimizing logical-qubit mapping and masking. "Original order" uses the circuit-derived mapping, while "random order" samples 10 random candidates and reports the best. (b) Native-measurement ratio versus masking in gross and two-gross codes, for both full-Pauli and I/Z-only operator sets. (c) Reduction in automorphism rounds from TSP-based scheduling, relative to the original gate order in [46]. (d) Qubit-recycling compression reduces logical-qubit counts for large protocols without increasing total measurements.

two-gross cases, where inter-module measurements are cleaner, the performance gap narrows and the preferred choice becomes workload-dependent: direct injection is attractive for high throughput, while pivot injection remains preferable when targeting the lowest output error.

G. Sensitivity Analysis in Logical Operation Error Rates

Finally, we perform a one-factor-at-a-time sensitivity study around the baseline logical error rates in Table I. We exclude automorphism operations, since their error rates are primarily set by code design and are weakly sensitive to the specific LPU design. For each factory configuration, we sweep one logical operation error parameter at a time (in-module or inter-module measurement), fix the others, and record the resulting change in p_{out}.

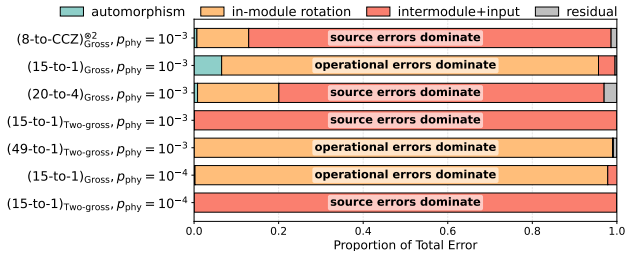

Fig. 10: Dominant error source by distillation protocol. Most factories are limited either by *operational errors* during distillation (e.g., in-module rotation errors) or *source errors* (e.g., input magic state errors or inter-module injection errors).

Factory	$p_{\text{phys}} = p_{\text{in}}$	τ_i	Space-time Volume	$p_{\text{out}}^{(\text{sim})}$
15-to-1$_{\text{gross}}$, direct	10^{-3}	2341	8.8×10^5	8.2×10^{-4}
15-to-1$_{\text{gross}}$, pivot	10^{-3}	6122	2.3×10^6	4.6×10^{-6}
15-to-1$_{\text{two-gross}}$, direct	10^{-3}	4145	3.0×10^6	1.1×10^{-8}
15-to-1$_{\text{two-gross}}$, pivot	10^{-3}	11249	8.3×10^6	1.0×10^{-8}
8-to-CCZ$_{\text{gross}}^{\otimes 2}$, direct	10^{-3}	725	1.7×10^5	2.5×10^{-3}
8-to-CCZ$_{\text{gross}}^{\otimes 2}$, pivot	10^{-3}	1570	5.9×10^5	9.8×10^{-5}
49-to-1$_{\text{two-gross}}$, direct	10^{-3}	63651	4.7×10^7	9.6×10^{-9}
49-to-1$_{\text{two-gross}}$, pivot	10^{-3}	70748	5.2×10^7	9.7×10^{-11}
15-to-1$_{\text{gross}}$, direct	10^{-4}	2303	8.7×10^5	2.1×10^{-8}
15-to-1$_{\text{gross}}$, pivot	10^{-4}	5999	2.3×10^6	4.2×10^{-10}

TABLE IV: Comparison between direct injection (Fig. 4(a),(c)) and pivot injection (Fig. 4(b)) across representative distillation protocols. Direct injection reduces distillation timesteps, while pivot injection suppresses low-fidelity inter-module measurement errors and typically achieves lower output error (detailed discussion in Sec. VII-F).

Figure 11 sweeps in-module and inter-module operation error rates and reports contours of constant output magic-state error. The knee of each contour marks the transition between in-module-dominated and inter-module-dominated regimes. Across the sweep, pivot injection is more robust to inter-module noise, shifting contours toward higher inter-module error rates at fixed output error. Overall, improving either error source helps, with the largest gains from better in-module measurement fidelity combined with pivot injection.

VIII. Discussion

A. Further Reducing Timesteps in BB Architectures

In Section VII, we saw that BB-based distillation schemes achieve strong qubit savings at the cost of larger depth τ_i. Here, we explore a simple knob that reduces the number of syndrome-extraction rounds to lower τ_i in MSD circuits.

As observed in Ref. [31], the two error modes for in-module measurements, measurement-outcome flips p_{meas} and logical memory errors p_{memory}, move in opposite directions as the number of rounds n changes: p_{meas} decreases roughly exponentially with n, while p_{memory} grows approximately linearly [3].

[3]The syndrome extraction rounds and logical error rates are taken from the results of [31], which is based on an LPU design that differs slightly from that in [27] but yields comparable logical error rates.

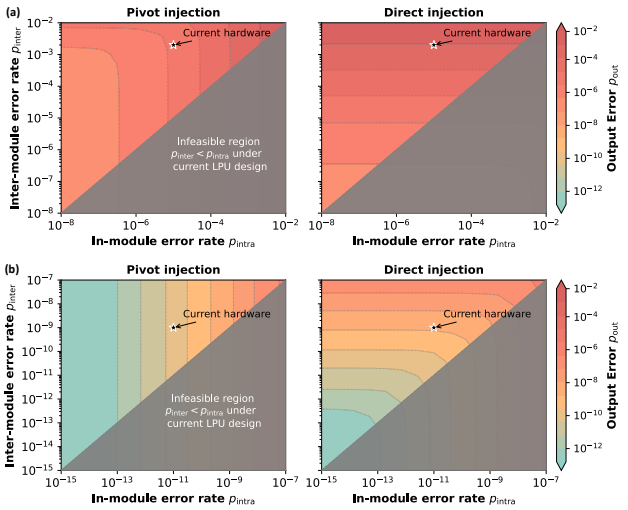

Fig. 11: Sensitivity of output magic-state error to in-module and inter-module operation error rates. (a) 15-to-1 protocol for the gross code. (b) 49-to-1 protocol for the two-gross code. Since inter-module operations use in-module measurements as subroutines, we consider the regime $p_{\text{inter}} \geq p_{\text{intra}}$; the gray region indicates excluded parameters. The current-hardware point corresponds to logical error rates derived at physical error rate $p = 10^{-3}$ [27].

Factory	Syndrome Rounds	p_{meas}	p_{memory}	τ_i	$p_{\text{out}}^{(\text{sim})}$
8-to-CCZ$_{\text{gross}}^{\otimes 2}$	$7 \to 5$	$10^{-3.5}$	$10^{-5.2}$	$1570 \to [1159]$	1.2×10^{-4}
15-to-1$_{\text{gross}}$	$7 \to 4$	$10^{-2.7}$	$10^{-5.5}$	$6122 \to [3808]$	3.5×10^{-6}
20-to-4$_{\text{gross}}$	$7 \to 5$	$10^{-3.5}$	$10^{-5.2}$	$3088 \to [2581]$	7.8×10^{-5}

TABLE V: Implementing magic-state distillation protocols in the gross code with fewer syndrome-extraction rounds. Entries of the form $x \to [y]$ denote the baseline timestep count and the reduced value at smaller n. This can speed up distillation with no or moderate impact on the output fidelity.

General-purpose circuits typically choose n near the balance point that minimizes total logical error. Distillation protocols, however, can tolerate some measurement flips, so their optimal n can be smaller. In Section VII, we benchmarked how the parameter λ, the ratio between measurement error and the total logical operation error rate, affects the overall output error rate. Here, we reduce the number of syndrome-extraction rounds, thereby introducing more but tolerable outcome-flip errors while suppressing memory errors.

Table V reports the resulting estimated timestep reductions (shown in brackets). In the gross-code configuration, reducing n from 7 to 4 for the 15-to-1 protocol cuts τ_i from 6122 to $[3808]$ while keeping the output error at $p_{\text{out}}^{(\text{sim})} \approx 3.5 \times 10^{-6}$. For 8-to-CCZ and 20-to-4, errors are already dominated by source errors, so decreasing n slightly worsens fidelity but still yields substantial savings in τ_i. In practice, operation-limited protocols can therefore trade a modest increase in p_{meas} for noticeably shorter distillation time.

B. Future Directions for Magic Factory Design and Fault-tolerant Computing in Bicycle Architecture

Future improvements can be grouped into three directions. First, increasing the fidelity of in-module and inter-module logical operations through improved LPU design would directly lower output magic-state error and increase attainable throughput. Second, improving decoder performance can substantially improve logical operation fidelity and may provide a dominant improvement in practice. Third, it is important to study the impact of reducing the number of syndrome-extraction rounds: fewer rounds can lower latency and space-time cost, but may increase measurement-induced failures; quantifying this tradeoff would make the discussion above more actionable.

IX. Conclusion

High-fidelity, resource-efficient magic-state distillation is critical for scalable fault-tolerant quantum computing. We introduced practical distillation factories on Bivariate Bicycle codes, which achieve low target error rates with competitive space–time costs while reducing the qubit footprint. When preceded by surface code cultivation, our protocols constitute compelling two-round factories for near-term devices. Looking ahead, our methodology is general and adaptive across protocols and hardware performance. As the bicycle architecture achieves lower logical error rates through better decoding, circuits, or devices, the factory design methodology in our paper delivers even lower output error rates with lower overhead, positioning BB-based magic state distillation as a robust building block for large-scale quantum platforms.

Acknowledgment

We thank Andrew Cross, Theodore J Yoder, Tomas Jochym-O'Connor, Shraddha Singh, Zhixin Song, Xiang Fang, Ming Wang, Yue Wu, Shuwen Kan, Sean Garner, Samuel Stein, Chenxu Liu, and Ang Li for fruitful discussions. This work is supported in part by the National Science Foundation (under awards CCF-2312754 and CCF-2338063), in part by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Center, Co-design Center for Quantum Advantage (C2QA) under Contract No. DE-SC0012704, in part by QuantumCT (under NSF Engines award ITE-2302908), in part by AFOSR MURI (FA9550-26-1-B036). ZH acknowledges support from the MIT Department of Mathematics, the MIT-IBM Watson AI Lab, and the NSF Graduate Research Fellowship Program under Grant No. 2141064. YD acknowledges partial support by Boehringer Ingelheim, and NSF NQVL-ERASE (under award OSI-2435244). External interest disclosure: YD is a scientific advisor to D-Wave Quantum, Inc.

References

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994, pp. 124–134.

[2] P. Shor, "Fault-tolerant quantum computation," in *Proceedings of 37th Conference on Foundations of Computer Science*, 1996, pp. 56–65. [Online]. Available: https://ieeexplore.ieee.org/document/548464

[3] A. Y. Kitaev, "Quantum computations: algorithms and error correction," *Russian Mathematical Surveys*, vol. 52, no. 6, p. 1191–1249, Dec 1997. [Online]. Available: http://dx.doi.org/10.1070/RM1997v052n06ABEH002155

[4] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical review A*, vol. 52, no. 4, p. R2493, 1995.

[5] A. Steane, "Multiple-particle interference and quantum error correction," *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 452, no. 1954, pp. 2551–2577, 1996. [Online]. Available: https://royalsocietypublishing.org/doi/10.1098/rspa.1996.0136

[6] A. R. Calderbank and P. W. Shor, "Good quantum error-correcting codes exist," *Physical Review A*, vol. 54, no. 2, p. 1098, 1996. [Online]. Available: https://journals.aps.org/pra/abstract/10.1103/PhysRevA.54.1098

[7] D. Gottesman, "Stabilizer codes and quantum error correction," Ph.D. dissertation, California Institute of Technology, 1997. [Online]. Available: https://thesis.library.caltech.edu/2900/

[8] D. Aharonov and M. Ben-Or, "Fault-tolerant quantum computation with constant error," in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC '97. New York, NY, USA: Association for Computing Machinery, 1997, p. 176–188. [Online]. Available: https://doi.org/10.1145/258533.258579

[9] S. B. Bravyi and A. Y. Kitaev, "Quantum codes on a lattice with boundary," *arXiv:quant-ph/9811052*, 1998. [Online]. Available: https://arxiv.org/abs/quant-ph/9811052

[10] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *Journal of Mathematical Physics*, vol. 43, no. 9, p. 4452–4505, Sep 2002. [Online]. Available: http://dx.doi.org/10.1063/1.1499754

[11] "Kitaev surface code," in *The Error Correction Zoo*, V. V. Albert and P. Faist, Eds., 2025. [Online]. Available: https://errorcorrectionzoo.org/c/surface

[12] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, Apr 2021. [Online]. Available: https://doi.org/10.22331/q-2021-04-15-433

[13] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," *arXiv preprint arXiv:2505.15917*, 2025.

[14] D. Gottesman, "Fault-Tolerant Quantum Computation with Constant Overhead," *Quantum Info. Comput.*, vol. 14, no. 15–16, p. 1338–1372, Nov 2014. [Online]. Available: https://dl.acm.org/doi/10.5555/2685179.2685184

[15] "Quantum LDPC (QLDPC) code," in *The Error Correction Zoo*, V. V. Albert and P. Faist, Eds., 2025. [Online]. Available: https://errorcorrectionzoo.org/c/qldpc

[16] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, no. 8005, p. 778–782, Mar 2024. [Online]. Available: http://dx.doi.org/10.1038/s41586-024-07107-7

[17] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Phys. Rev. A*, vol. 86, p. 032324, Sep 2012. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.86.032324

[18] A. G. Fowler and C. Gidney, "Low overhead quantum computation using lattice surgery," 2018. [Online]. Available: https://arxiv.org/abs/1808.06709

[19] D. Litinski, "A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery," *Quantum*, vol. 3, p. 128, Mar 2019. [Online]. Available: http://dx.doi.org/10.22331/q-2019-03-05-128

[20] H. Zhou, C. Zhao, M. Cain, D. Bluvstein, C. Duckering, H.-Y. Hu, S.-T. Wang, A. Kubica, and M. D. Lukin, "Algorithmic Fault Tolerance for Fast Quantum Computing," 2024. [Online]. Available: https://arxiv.org/abs/2406.17653

[21] O. Fawzi, A. Grospellier, and A. Leverrier, "Constant Overhead Quantum Fault-Tolerance with Quantum Expander Codes," in *2018 IEEE 59th Annu. Symp. Found. Comput. Scie. (FOCS)*, vol. 64, no. 1. ACM New York, NY, USA, Oct 2018, pp. 106–114. [Online]. Available: https://doi.org/10.1109%2Ffocs.2018.00076

[22] S. Tamiya, M. Koashi, and H. Yamasaki, "Polylog-time-and constant-space-overhead fault-tolerant quantum computation with quantum low-density parity-check codes," *arXiv preprint arXiv:2411.03683*, 2024. [Online]. Available: https://arxiv.org/abs/2411.03683

[23] Q. T. Nguyen, "Good Binary Quantum Codes with Transversal CCZ Gate," in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, ser. STOC '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 697–706. [Online]. Available: https://doi.org/10.1145/3717823.3718186

[24] Z. He, Q. T. Nguyen, and C. A. Pattison, "Composable Quantum Fault-Tolerance," 2025. [Online]. Available: https://arxiv.org/abs/2508.08246

[25] G. Zhang, Y. Zhu, and Y. Li, "Accelerating Fault-Tolerant Quantum Computation with Good qLDPC Codes," *arXiv preprint arXiv:2510.19442*, 2025.

[26] Z. He, A. Cowtan, D. J. Williamson, and T. J. Yoder, "Extractors: QLDPC Architectures for Efficient Pauli-Based Computation," 2025. [Online]. Available: https://arxiv.org/abs/2503.10390

[27] T. J. Yoder, E. Schoute, P. Rall, E. Pritchett, J. M. Gambetta, A. W. Cross, M. Carroll, and M. E. Beverland, "Tour de gross: A modular quantum computer based on bivariate bicycle codes," *arXiv preprint arXiv:2506.03094*, 2025. [Online]. Available: https://arxiv.org/abs/2506.03094

[28] D. Gottesman and I. L. Chuang, "Demonstrating the viability of universal quantum computation using teleportation and single-qubit operations," *Nature*, vol. 402, no. 6760, p. 390–393, Nov 1999. [Online]. Available: http://dx.doi.org/10.1038/46503

[29] S. Bravyi and A. Kitaev, "Universal quantum computation with ideal Clifford gates and noisy ancillas," *Phys. Rev. A*, vol. 71, p. 022316, Feb 2005. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.71.022316

[30] C. Gidney, N. Shutty, and C. Jones, "Magic state cultivation: growing T states as cheap as CNOT gates," 2024. [Online]. Available: https://arxiv.org/abs/2409.17595

[31] A. Cross, Z. He, P. Rall, and T. Yoder, "Improved QLDPC Surgery: Logical Measurements and Bridging Codes," 2024. [Online]. Available: https://arxiv.org/abs/2407.18393

[32] D. J. Williamson and T. J. Yoder, "Low-overhead fault-tolerant quantum computation by gauging logical operators," *arXiv preprint arXiv:2410.02213*, 2024. [Online]. Available: https://arxiv.org/abs/2410.02213

[33] B. Ide, M. G. Gowda, P. J. Nadkarni, and G. Dauphinais, "Fault-Tolerant Logical Measurements via Homological Measurement," *Phys. Rev. X*, vol. 15, p. 021088, Jun 2025. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevX.15.021088

[34] E. Swaroop, T. Jochym-O'Connor, and T. J. Yoder, "Universal adapters between quantum LDPC codes," 2025. [Online]. Available: https://arxiv.org/abs/2410.03628

[35] P. Panteleev and G. Kalachev, "Degenerate Quantum LDPC Codes With Good Finite Length Performance," *Quantum*, vol. 5, p. 585, Nov 2021. [Online]. Available: https://doi.org/10.22331/q-2021-11-22-585

[36] Q. Xu, J. P. Bonilla Ataides, C. A. Pattison, N. Raveendran, D. Bluvstein, J. Wurtz, B. Vasić, M. D. Lukin, L. Jiang, and H. Zhou, "Constant-overhead fault-tolerant quantum computation with reconfigurable atom arrays," *Nature Physics*, vol. 20, no. 7, p. 1084–1090, Apr 2024. [Online]. Available: http://dx.doi.org/10.1038/s41567-024-02479-z

[37] H.-K. Lin and L. P. Pryadko, "Quantum two-block group algebra codes," *Physical Review A*, vol. 109, no. 2, p. 022407, 2024.

[38] N. P. Breuckmann and S. Burton, "Fold-Transversal Clifford Gates for Quantum Codes," *Quantum*, vol. 8, p. 1372, Jun. 2024. [Online]. Available: http://dx.doi.org/10.22331/q-2024-06-13-1372

[39] Q. Xu, H. Zhou, D. Bluvstein, M. Cain, M. Kalinowski, J. Preskill, M. D. Lukin, and N. Maskara, "Batched high-rate logical operations for quantum LDPC codes," 2025. [Online]. Available: https://arxiv.org/abs/2510.06159

[40] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario, "The future of quantum computing with superconducting qubits," *Journal of Applied Physics*, vol. 132, no. 16, Oct 2022. [Online]. Available: http://dx.doi.org/10.1063/5.0082975

[41] P. Team, "A manufacturable platform for photonic quantum computing," *Nature*, pp. 1–3, 2025.

[42] H. Aghaee Rad, T. Ainsworth, R. Alexander, B. Altieri, M. Askarani, R. Baby, L. Banchi, B. Baragiola, J. Bourassa, R. Chadwick *et al.*, "Scaling and networking a modular photonic quantum computer," *Nature*, pp. 1–8, 2025.

[43] D. Bluvstein, S. J. Evered, A. A. Geim, S. H. Li, H. Zhou, T. Manovitz, S. Ebadi, M. Cain, M. Kalinowski, D. Hangleiter, J. P. Bonilla Ataides, N. Maskara, I. Cong, X. Gao, P. Sales Rodriguez, T. Karolyshyn, G. Semeghini, M. J. Gullans, M. Greiner, V. Vuletić, and M. D. Lukin, "Logical quantum processor based on reconfigurable atom arrays," *Nature*, vol. 626, no. 7997, p. 58–65, Dec. 2023. [Online]. Available: http://dx.doi.org/10.1038/s41586-023-06927-3

[44] S. Bravyi, G. Smith, and J. A. Smolin, "Trading Classical and Quantum Computational Resources," *Physical Review X*, vol. 6, no. 2, Jun 2016. [Online]. Available: http://dx.doi.org/10.1103/PhysRevX.6.021043

[45] L. Z. Cohen, I. H. Kim, S. D. Bartlett, and B. J. Brown, "Low-overhead fault-tolerant quantum computing using long-range connectivity," *Science Advances*, vol. 8, no. 20, May 2022. [Online]. Available: http://dx.doi.org/10.1126/sciadv.abn1717

[46] D. Litinski, "Magic State Distillation: Not as Costly as You Think," *Quantum*, vol. 3, p. 205, Dec. 2019. [Online]. Available: http://dx.doi.org/10.22331/q-2019-12-02-205

[47] S. Bravyi and J. Haah, "Magic-state distillation with low overhead," *Physical Review A*, vol. 86, no. 5, Nov. 2012. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.86.052329

[48] K. Sahay, P.-K. Tsai, K. Chang, Q. Su, T. B. Smith, S. Singh, and S. Puri, "Fold-transversal surface code cultivation," 2025. [Online]. Available: https://arxiv.org/abs/2509.05212

[49] Y. Li, "A magic state's fidelity can be superior to the operations that created it," *New Journal of Physics*, vol. 17, no. 2, p. 023037, Feb. 2015. [Online]. Available: http://dx.doi.org/10.1088/1367-2630/17/2/023037

[50] N. Yoshioka, A. Seif, A. Cross, and A. Javadi-Abhari, "Transversal gates for probabilistic implementation of multi-qubit Pauli rotations," 2025. [Online]. Available: https://arxiv.org/abs/2510.08290